

WEEK 3 – ASSIGNMENT

Objective : Use Subqueries, CTEs, and Window Functions to analyze sales data from the Superstore dataset.

Step 1: Setup Data –

Import the Superstore dataset into a table named superstore_raw. Create these 3 tables from it:

Customers

Orders

Products

Insert data into these tables using SELECT DISTINCT.

1. Customers

```
89  -- Verify customer Data
90 •  SELECT * FROM customers LIMIT 5;
```

customer_id	customer_name	segment	country	city	state	postal_code	region
CG-12520	Claire Gute	Consumer	United States	Henderson	Kentucky	42420	South
DV-13045	Darrin Van Huff	Corporate	United States	Los Angeles	California	90036	West
SO-20335	Sean O'Donnell	Consumer	United States	Fort Lauderdale	Florida	33311	South
BH-11710	Brosina Hoffman	Consumer	United States	Los Angeles	California	90032	West
AA-10480	Andrew Allen	Consumer	United States	Concord	North Carolina	28027	South

2. Products

```
113  -- Verify product Data
114 •  SELECT * FROM products LIMIT 10;
```

product_id	category	sub_category	product_name
FUR-BO-10001798	Furniture	Bookcases	Bush Somerset Collection Bookcase
FUR-CH-10000454	Furniture	Chairs	Hon Deluxe Fabric Upholstered Stacking Chairs,...
OFF-LA-10000240	Office Supplies	Labels	Self-Adhesive Address Labels for Typewriters b...
FUR-TA-10000577	Furniture	Tables	Bretford CR4500 Series Slim Rectangular Table
OFF-ST-10000760	Office Supplies	Storage	Eldon Fold 'N Roll Cart System
FUR-FU-10001487	Furniture	Furnishings	Eldon Expressions Wood and Plastic Desk Acces...
OFF-AR-10002833	Office Supplies	Art	Newell 322
TEC-PH-10002275	Technology	Phones	Mitel 5320 IP Phone VoIP phone
OFF-BI-10003910	Office Supplies	Binders	DXL Angle-View Binders with Locking Rings by S...
OFF-AP-10002892	Office Supplies	Appliances	Belkin F5C206VTEL 6 Outlet Surge

products 7 x

3. Orders

```
150 -- Verify order records
151 • SELECT * FROM orders LIMIT 10;
```

	order_id	customer_id	product_id	order_date	ship_date	ship_mode	quantity	sales	discount	profit
▶	CA-2016-152156	CG-12520	FUR-BO-10001798	11/8/2016	11/11/2016	Second Class	2	261.96	0	41.9136
	CA-2016-152156	CG-12520	FUR-CH-10000454	11/8/2016	11/11/2016	Second Class	3	731.94	0	219.582
	CA-2016-138688	DV-13045	OFF-LA-10000240	6/12/2016	6/16/2016	Second Class	2	14.62	0	6.8714
	US-2015-108966	SO-20335	FUR-TA-10000577	10/11/2015	10/18/2015	Standard Class	5	957.5775	0.45	-383.031
	US-2015-108966	SO-20335	OFF-ST-10000760	10/11/2015	10/18/2015	Standard Class	2	22.368	0.2	2.5164
	CA-2014-115812	BH-11710	FUR-FU-10001487	6/9/2014	6/14/2014	Standard Class	7	48.86	0	14.1694
	CA-2014-115812	BH-11710	OFF-AR-10002833	6/9/2014	6/14/2014	Standard Class	4	7.28	0	1.9656
	CA-2014-115812	BH-11710	TEC-PH-10002275	6/9/2014	6/14/2014	Standard Class	6	907.152	0.2	90.7152
	CA-2014-115812	BH-11710	OFF-BI-10003910	6/9/2014	6/14/2014	Standard Class	3	18.504	0.2	5.7825
	CA-2014-115812	BH-11710	OFF-AP-10002892	6/9/2014	6/14/2014	Standard Class	5	114.9	0	34.47

Step 2 : Perform Required Queries—

1. Find all orders where sales are greater than the average sales. (Subquery)

```
162 • SELECT * FROM orders
163 WHERE CAST(sales AS DECIMAL(10,2)) >
164 (
165     SELECT AVG(CAST(sales AS DECIMAL(10,2)))
166     FROM orders
167 );
```

	order_id	customer_id	product_id	order_date	ship_date	ship_mode	quantity	sales	discount	profit
▶	CA-2016-152156	CG-12520	FUR-BO-10001798	11/8/2016	11/11/2016	Second Class	2	261.96	0	41.9136
	CA-2016-152156	CG-12520	FUR-CH-10000454	11/8/2016	11/11/2016	Second Class	3	731.94	0	219.582
	US-2015-108966	SO-20335	FUR-TA-10000577	10/11/2015	10/18/2015	Standard Class	5	957.5775	0.45	-383.031
	CA-2014-115812	BH-11710	TEC-PH-10002275	6/9/2014	6/14/2014	Standard Class	6	907.152	0.2	90.7152
	CA-2014-115812	BH-11710	FUR-TA-10001539	6/9/2014	6/14/2014	Standard Class	9	1706.184	0.2	85.3092
	CA-2014-115812	BH-11710	TEC-PH-10002033	6/9/2014	6/14/2014	Standard Class	4	911.424	0.2	68.3568
	CA-2016-161389	IM-15070	OFF-BI-10003656	12/5/2016	12/10/2016	Standard Class	3	407.976	0.2	132.5922
	CA-2014-105893	PK-19075	OFF-ST-10004186	11/11/2014	11/18/2014	Standard Class	6	665.88	0	13.3176
	CA-2015-106320	EB-13870	FUR-TA-10000577	9/25/2015	9/30/2015	Standard Class	3	1044.63	0	240.2649
	US-2015-150630	TB-21520	FUR-BO-10004834	9/17/2015	9/21/2015	Standard Class	7	3083.43	0.5	-1665.0522
	CA-2016-117590	GH-14485	TEC-PH-10004977	12/8/2016	12/10/2016	First Class	7	1097.544	0.2	123.4737
	CA-2015-117415	SN-20710	FUR-BO-10002545	12/27/2015	12/31/2015	Standard Class	3	532.3992	0.32	-46.9764
	CA-2015-117415	SN-20710	TEC-PH-10000486	12/27/2015	12/31/2015	Standard Class	4	371.168	0.2	41.7564

2. Find the highest sales order for each customer. (Subquery)

```

172 • SELECT * FROM orders o
173 WHERE CAST(o.sales AS DECIMAL(10,2)) =
174 (
175     SELECT MAX(CAST(i.sales AS DECIMAL(10,2)))
176     FROM orders i
177     WHERE i.customer_id = o.customer_id
178 )

```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: [↗](#)

order_id	customer_id	product_id	order_date	ship_date	ship_mode	quantity	sales	discount	profit
CA-2016-152156	CG-12520	FUR-CH-10000454	11/8/2016	11/11/2016	Second Class	3	731.94	0	219.582
US-2015-108966	SO-20335	FUR-TA-10000577	10/11/2015	10/18/2015	Standard Class	5	957.5775	0.45	-383.031
CA-2014-115812	BH-11710	FUR-TA-10001539	6/9/2014	6/14/2014	Standard Class	9	1706.184	0.2	85.3092
CA-2015-106320	EB-13870	FUR-TA-10000577	9/25/2015	9/30/2015	Standard Class	3	1044.63	0	240.2649
US-2015-150630	TB-21520	FUR-BO-10004834	9/17/2015	9/21/2015	Standard Class	7	3083.43	0.5	-1665.0522
CA-2016-117590	GH-14485	TEC-PH-10004977	12/8/2016	12/10/2016	First Class	7	1097.544	0.2	123.4737
CA-2015-117415	SN-20710	FUR-BO-10002545	12/27/2015	12/31/2015	Standard Class	3	532.3992	0.32	-46.9764
CA-2016-105816	JM-15265	TEC-PH-10002447	12/11/2016	12/17/2016	Standard Class	5	1029.95	0	298.6855
CA-2016-111682	TB-21055	FUR-CH-10003968	6/17/2016	6/18/2016	First Class	5	319.41	0.1	7.098
CA-2014-106376	RS-11590	OFF-AD-10002671	12/5/2014	12/10/2014	Standard Class	8	1113.074	0.2	111.3074

orders 11 x

3. Calculate total sales for each customer. (CTE)

```

183
184 • WITH customer_sales AS (
185     SELECT
186         customer_id,
187         SUM(CAST(sales AS DECIMAL(10,2))) AS total_sales
188     FROM orders
189     GROUP BY customer_id
190 )
191
192 SELECT *
193 FROM customer_sales;

```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: [↗](#)

customer_id	total_sales
CG-12520	1148.78
DV-13045	1119.48
SO-20335	2602.58
BH-11710	6255.34
AA-10480	1790.51
IM-15070	4930.49
HP-14815	1990.31
PK-19075	8646.93
AG-10270	2582.90
ZD-21925	1493.94
KB-16585	2744.74
SF-20065	763.54

4. Find customers whose total sales are above average. (CTE + Subquery)

```

199 WITH customer_sales AS (
200     SELECT
201         customer_id,
202         SUM(CAST(sales AS DECIMAL(10,2))) AS total_sales
203     FROM orders
204     GROUP BY customer_id
205 )
206 SELECT *
207 FROM customer_sales
208 WHERE total_sales >
209 (
210     SELECT AVG(total_sales)
211     FROM customer_sales
212 );

```

customer_id	total_sales
BH-11710	6255.34
IM-15070	4930.49
PK-19075	8646.93
TB-21520	4737.48
MA-17560	4299.16
SN-20710	3127.97
LC-16930	4492.94
RA-19885	3832.32
ES-14080	4657.93
KM-16720	4909.47

5. Rank all customers based on total sales. (Window Function)

```

217 SELECT
218     customer_id,
219     SUM(CAST(sales AS DECIMAL(10,2))) AS total_sales,
220     RANK() OVER (
221         ORDER BY SUM(CAST(sales AS DECIMAL(10,2))) DESC
222     ) AS sales_rank
223 FROM orders
224 GROUP BY customer_id;

```

customer_id	total_sales	sales_rank
SM-20320	25043.07	1
TC-20980	19052.22	2
RB-19360	15117.35	3
TA-21385	14595.62	4
AB-10105	14473.57	5
KL-16645	14175.23	6
SC-20095	14142.34	7
HL-15040	12873.30	8
SE-20110	12209.44	9
CC-12370	12129.08	10
TS-21370	11891.75	11
GT-14710	11820.12	12
BM-11140	11789.64	13
SV-20365	11470.94	14
CJ-12010	11164.97	15
CL-12565	10880.56	16

6. Assign row numbers to each order within a customer. (Window Function + PARTITION BY)

```

231 • SELECT *,
232     ROW_NUMBER() OVER
233     (
234         PARTITION BY customer_id
235         ORDER BY CAST(sales AS DECIMAL(10,2)) DESC
236     ) AS row_num
237 FROM orders;

```

order_id	customer_id	product_id	order_date	ship_date	ship_mode	quantity	sales	discount	profit	row_num
CA-2016-103982	AA-10315	OFF-SU-10000151	3/3/2016	3/8/2016	Standard Class	3	3930.072	0.2	-786.0144	1
CA-2014-128055	AA-10315	OFF-BI-10004390	3/31/2014	4/5/2014	Standard Class	2	673.568	0.2	252.588	2
CA-2016-103982	AA-10315	TEC-PH-10000895	3/3/2016	3/8/2016	Standard Class	3	431.976	0.2	32.3982	3
CA-2017-147039	AA-10315	OFF-AP-10000576	6/29/2017	7/4/2017	Standard Class	3	362.94	0	90.735	4
CA-2014-128055	AA-10315	OFF-AP-10002765	3/31/2014	4/5/2014	Standard Class	2	52.98	0	14.8344	5
CA-2016-103982	AA-10315	TEC-AC-10002857	3/3/2016	3/8/2016	Standard Class	7	41.72	0.2	5.7365	6
CA-2015-121391	AA-10315	OFF-ST-10001590	10/4/2015	10/7/2015	First Class	2	26.96	0	7.0096	7
CA-2014-138100	AA-10315	OFF-PA-10000349	9/15/2014	9/20/2014	Standard Class	3	14.94	0	7.0218	8
CA-2014-138100	AA-10315	FUR-FU-10002456	9/15/2014	9/20/2014	Standard Class	2	14.56	0	6.2608	9
CA-2017-147039	AA-10315	OFF-BI-10004654	6/29/2017	7/4/2017	Standard Class	2	11.54	0	5.77	10

7. Display top 3 customers based on total sales. (Window Function)

```

243 • SELECT *
244 FROM (
245     SELECT
246         customer_id,
247         SUM(CAST(sales AS DECIMAL(10,2))) AS total_sales,
248         RANK() OVER (
249             ORDER BY SUM(CAST(sales AS DECIMAL(10,2))) DESC
250         ) AS sales_rank
251     FROM orders
252     GROUP BY customer_id
253 ) t
254 WHERE sales_rank <= 3;

```

customer_id	total_sales	sales_rank
SM-20320	25043.07	1
TC-20980	19052.22	2
RB-19360	15117.35	3

Final Combined Query (JOIN + CTE + Window Function together)

- Customer Name
- Total Sales
- Rank

```
261 WITH customer_sales AS (  
262     SELECT  
263         customer_id,  
264         SUM(CAST(sales AS DECIMAL(10,2))) AS total_sales  
265     FROM orders  
266     GROUP BY customer_id  
267 )  
268  
269 SELECT  
270     c.customer_name,  
271     cs.total_sales,  
272     RANK() OVER (  
273         ORDER BY cs.total_sales DESC  
274     ) AS sales_rank  
275 FROM customer_sales cs  
276 JOIN customers c  
277     ON cs.customer_id = c.customer_id;
```

< Result Grid Filter Rows: Export: Wrap Cell Content: Fetch rows:

	customer_name	total_sales	sales_rank
▶	Sean Miller	25043.07	1
	Sean Miller	25043.07	1
	Sean Miller	25043.07	1
	Sean Miller	25043.07	1
	Sean Miller	25043.07	1
	Tamara Chand	19052.22	6
	Tamara Chand	19052.22	6
	Tamara Chand	19052.22	6

Result 18 x

MINI PROJECT – CUSTOMER SALES INSIGHTS

1. Who are the top 5 customers?

```
287 WITH customer_sales AS (  
288     SELECT  
289         customer_id,  
290         SUM(CAST(sales AS DECIMAL(10,2))) AS total_sales  
291     FROM orders  
292     GROUP BY customer_id  
293 )  
294 SELECT * FROM (  
295     SELECT *,  
296         RANK() OVER (ORDER BY total_sales DESC) AS rnk  
297     FROM customer_sales  
298 ) t  
299 WHERE rnk <= 5;
```

customer_id	total_sales	rnk
SM-20320	25043.07	1
TC-20980	19052.22	2
RB-19360	15117.35	3
TA-21385	14595.62	4
AB-10105	14473.57	5

2. Who are the bottom 5 customers ?

```
305 WITH customer_sales AS (  
306     SELECT  
307         customer_id,  
308         SUM(CAST(sales AS DECIMAL(10,2))) AS total_sales  
309     FROM orders  
310     GROUP BY customer_id  
311 )  
312 SELECT * FROM (  
313     SELECT *,  
314         RANK() OVER (ORDER BY total_sales ASC) AS rnk  
315     FROM customer_sales  
316 ) t  
317 WHERE rnk <= 5;  
318
```

customer_id	total_sales	rnk
TS-21085	4.84	1
LD-16855	5.30	2
CJ-11875	16.52	3
MG-18205	16.74	4
RS-19870	22.33	5

3. Which customers made only one order?

```
323 • SELECT
324     customer_id,
325     COUNT(order_id) AS total_orders
326 FROM orders
327 GROUP BY customer_id
328 HAVING COUNT(order_id) = 1;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: [IA](#)

	customer_id	total_orders
▶	JR-15700	1
	AO-10810	1
	LD-16855	1
	CJ-11875	1
	RE-19405	1

Result 21 x

4. Which customers have above-average sales?

```
334 • WITH customer_sales AS (
335     SELECT
336         customer_id,
337         SUM(CAST(sales AS DECIMAL(10,2))) AS total_sales
338     FROM orders
339     GROUP BY customer_id
340 )
341 SELECT *
342 FROM customer_sales
343 WHERE total_sales >
344 (
345     SELECT AVG(total_sales)
346     FROM customer_sales
347 );
348
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: [IA](#)

	customer_id	total_sales
▶	BH-11710	6255.34
	IM-15070	4930.49
	PK-19075	8646.93
	TB-21520	4737.48
	MA-17560	4299.16
	SN-20710	3127.97
	LC-16930	4492.94
	RA-19885	3832.32

Result 22 x

5. What is the highest order value per customer?

```
351  -----
352  •  SELECT
353      customer_id,
354      MAX(CAST(sales AS DECIMAL(10,2))) AS highest_order
355  FROM orders
356  GROUP BY customer_id;
```

<

Result Grid |  Filter Rows: | Export:  | Wrap Cell Content: 

	customer_id	highest_order
▶	CG-12520	731.94
	DV-13045	721.88
	SO-20335	957.58
	BH-11710	1706.18
	AA-10480	479.97
	IM-15070	1819.86
	HP-14815	1242.90
	PK-19075	4305.55

Result 23 x